

CloudPulse – Presentation Script

What to say on every slide and click, with simple explanations and likely questions

Presenter: Cecil Baraik • Roll 2025-BSCIT/004A

Deck: CloudPulse_Presentation.pptx (15 slides)

Time: about 15 minutes of talk + 5–7 minutes of live demo

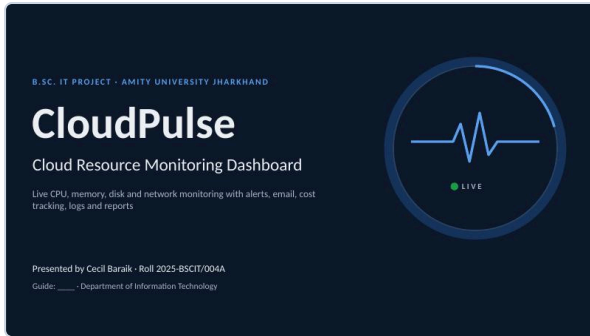
How to use this script

Each slide has four parts:

- **On the slide:** what the audience sees.
- **What to say:** your spoken script. The orange label tells you which click shows which part, matching the animations in the PowerPoint.
- **Understand it yourself:** the technical words explained simply, so you can explain them in your own words.
- **If they ask:** likely questions on that slide, with short answers.

Presenting tips

- Practise with the PowerPoint in Slide Show (F5). A click (or right arrow) shows the next item; the cues in this script tell you what appears on each click.
- Total talk is about 15 minutes, plus 5–7 minutes of demo. If time is short, say one sentence per click.
- Do not read from the slide. Look at the audience and use the slide as your reminder.
- If you do not know an answer, say: "I have not implemented that yet, but I would do it by ..." That is better than guessing.
- Fill in the guide's name on slide 1 and your GitHub username on the last slide before presenting.



SLIDE 1 OF 15

CloudPulse – Cloud Resource Monitoring Dashboard

45 sec

On the slide: Title, subtitle, a short description, your name and roll number, the guide line, and a spinning ring with a heartbeat line and a blinking LIVE dot.

What to say

Slide opens (animates by itself) Good morning everyone. I am Cecil Baraik, from B.Sc. IT, roll number 2025-BSCIT/004A. My project is called **CloudPulse**, a Cloud Resource Monitoring Dashboard.

In simple words, CloudPulse is a website that watches computers running in the cloud and shows their health live. Every 3 seconds it shows how much CPU, memory, disk and network each server is using.

If something goes wrong, for example CPU goes above 95 percent, it immediately shows an alert, and for serious problems it also sends an email. It also tracks how much the servers cost, shows live logs, and can export reports as PDF and CSV.

The backend is written in **Java with Spring Boot**, the frontend is **React**, and all data is stored in **MySQL**. The heartbeat on the right is a symbol of what the project does: it keeps checking the pulse of the servers.

Understand it yourself

Cloud resource: A server (a computer) or service rented from a company like Amazon (AWS), Microsoft (Azure) or Google (GCP) instead of buying your own hardware.

Monitoring: Continuously measuring how a server is doing: is it overloaded, running out of memory, out of disk space, or switched off?

Dashboard: One screen that shows many important numbers and charts together, like the dashboard of a car.

If they ask

Q. Why did you choose this project?

Every company that uses the cloud needs monitoring. It let me use many real Java topics in one project: REST APIs, a database, security, scheduling, real-time WebSocket and email.

Q. Is it a desktop application or a web application?

A web application. The Java server runs on my machine and anyone on the network can open it in a browser.

CONTENTS

Click a topic to jump to it

01	Introduction	The problem and the eight objectives
02	Design	Tech stack, architecture, live update flow, database
03	Features	Monitoring, alerts and email, costs, logs, security
04	Results	Testing, limitations and future scope
05	Demo and questions	Live demo, then Q&A

CloudPulse - Cloud Resource Monitoring Dashboard

2

SLIDE 2 OF 15

Contents

20 sec

On the slide: Five clickable topics: Introduction, Design, Features, Results, Demo and questions. Each item slides in by itself.

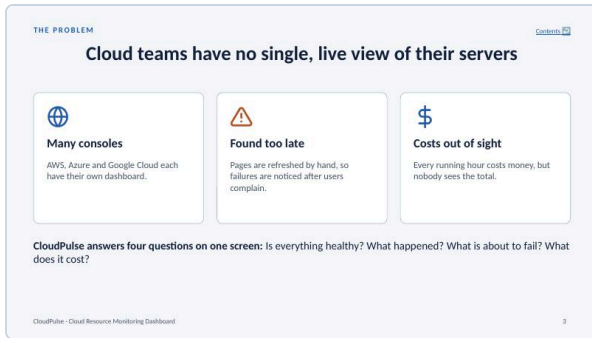
What to say

Slide opens (items appear by themselves) This is the plan of my presentation. First the **introduction**: the problem and my objectives. Then the **design**: technology, architecture, how live updates work and the database.

Then the **features**: monitoring, alerts and email, costs, logs and security. Then the **results**: testing and future scope. And finally a **live demo** and your questions.

Understand it yourself

Clickable topics: Each topic is a link. If an examiner asks to see a section again, click it here to jump straight there. Every other slide has a small "Contents" link at the top right to come back to this slide.



SLIDE 3 OF 15

The problem

1 min

On the slide: Three cards: Many consoles, Found too late, Costs out of sight. Then one line with CloudPulse's answer.

What to say

Click 1 – Many consoles Companies today often use more than one cloud: some servers on AWS, some on Azure, some on Google Cloud. Each one has its own separate website, or console. So to check everything, an engineer has to open many tabs and log in again and again.

Click 2 – Found too late Many teams still check servers by refreshing pages by hand. So a problem, like a server running out of memory, is often noticed only when users start complaining that the website is slow or down.

Click 3 – Costs out of sight Cloud servers are charged by the hour. If someone forgets to stop a server, it keeps costing money, and nobody sees the total until the bill arrives.

Click 4 – The answer So CloudPulse answers four questions on one screen: **Is everything healthy? What happened? What is about to fail? And what does it cost?**

Understand it yourself

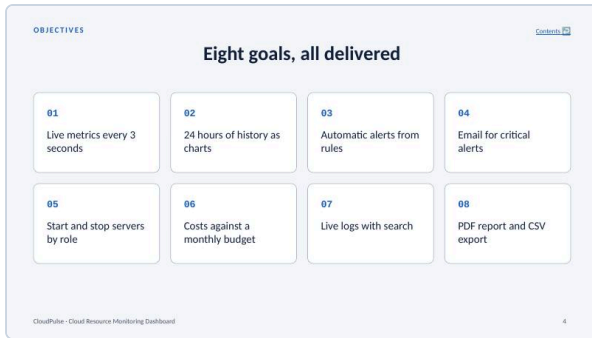
Console: The control website of a cloud provider, for example the AWS Management Console.

Why "found too late" matters: Every minute a server is down can mean lost customers. Monitoring with alerts finds the problem before users do.

If they ask

Q. Don't AWS and Azure already have monitoring?

Yes, AWS CloudWatch and Azure Monitor exist, but each one only shows its own cloud. CloudPulse shows all of them together, plus costs, in one simple screen.



SLIDE 4 OF 15

Objectives

1 min

On the slide: Eight numbered goal cards in two rows of four.

What to say

Click 1 – Goals 1 to 4 I set eight objectives. One: show live metrics every 3 seconds. Two: keep 24 hours of history and show it as charts. Three: raise alerts automatically using rules. Four: send an email for critical alerts.

Click 2 – Goals 5 to 8 Five: let users start and stop servers, depending on their role. Six: track costs against a monthly budget. Seven: show live logs with search. And eight: export a PDF report and CSV files.

All eight objectives are implemented and tested, and I will show each of them in the demo.

Understand it yourself

Metric: One measured value, such as CPU 45% or memory 70%.

Role: The type of user. An **Admin** can change things; a **Viewer** can only look.

CSV: Comma-Separated Values: a simple table file that opens in Excel.

If they ask

Q. Which objective was the hardest?

Real-time updates (objective 1), because the server must push data to every open browser at the same time. I solved it with WebSocket.

TECHNOLOGY STACK

Java on the server, React in the browser

Layer	Technology
Backend	Java 21, Spring Boot 3.5, Maven
Live updates	WebSocket with STOMP, pushed every 3 seconds
Database	MySQL 8, Spring Data JPA, Hibernate
Security	Spring Security, JWT tokens, BCrypt passwords
Real metrics	OSHI library (CPU, RAM, disk, network, uptime)
Reports and email	OpenPDF, Spring Mail with Gmail SMTP
Frontend	React 19, Vite, Tailwind CSS, Recharts, framer-motion
Tools	IntelliJ IDEA, Git and GitHub, Chrome

CloudPulse - Cloud Resource Monitoring Dashboard

SLIDE 5 OF 15

Technology stack

1 min

On the slide: A table: Backend, Live updates, Database, Security, Real metrics, Reports and email, Frontend, Tools.

What to say

Click – table appears These are the technologies I used. The **backend** is Java 21 with Spring Boot 3.5, built with Maven.

Live updates use WebSocket with STOMP, so the server pushes new data every 3 seconds. The **database** is MySQL 8, accessed through Spring Data JPA and Hibernate, so I write Java classes instead of SQL.

Security uses Spring Security with JWT tokens, and passwords are stored as BCrypt hashes. For **real metrics** of my own PC I used the OSHI library. **Reports** use OpenPDF and **emails** use Spring Mail with Gmail.

The **frontend** is React with Vite, Tailwind CSS for design and Recharts for charts. I developed in IntelliJ IDEA and used Git and GitHub for version control.

Understand it yourself

Spring Boot: A Java framework that makes it fast to build a web server. It has a built-in Tomcat server, so the app runs with one command.

JPA / Hibernate: An ORM (Object-Relational Mapping): it converts Java objects into database rows and back. I write `repository.save(alert)` instead of an INSERT query.

WebSocket: A connection that stays open between browser and server, so the server can send data any time. Normal HTTP only answers when the browser asks.

STOMP: A simple messaging format on top of WebSocket. It lets the browser subscribe to topics like `/topic/live`, `/topic/alerts` and `/topic/logs`.

React: A JavaScript library for building user interfaces from components. When data changes, only the changed part of the page updates.

Maven: A build tool that downloads libraries (dependencies) listed in `pom.xml` and builds the project.

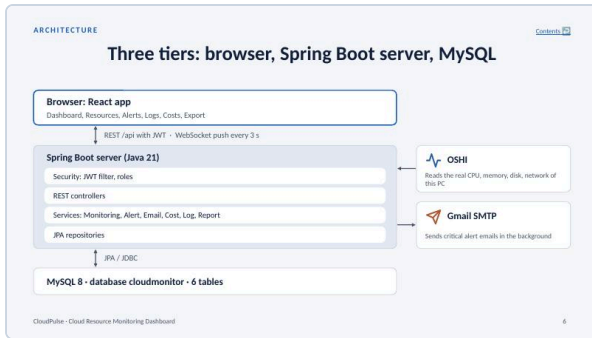
If they ask

Q. Why React and not JSP, JavaFX or Swing?

React gives smooth live charts in any browser, on phone or laptop, without installing anything. JavaFX and Swing are desktop-only. The core logic is still all Java.

Q. Why Spring Boot?

It is the industry standard for Java web services: security, database, scheduling, WebSocket and email are all available as ready modules.



SLIDE 6 OF 15

Architecture

1.5 min

On the slide: Three tiers: the React app in the browser, the Spring Boot server (security, controllers, services, repositories), MySQL, plus OSHI and Gmail SMTP on the side.

What to say

Click 1 – Browser My project has a three-tier architecture. The first tier is the **browser**, where the React app runs. It has pages for the dashboard, resources, alerts, logs, costs and export.

Click 2 – Spring Boot server The browser talks to the **Spring Boot server** in two ways. For normal actions, like login or stopping a server, it sends REST API calls with a JWT token. For live data, the server pushes updates over WebSocket every 3 seconds.

Inside the server, every request goes through layers: first the **security filter** checks the JWT token, then a **controller** receives the request, a **service** does the business logic, and a **repository** talks to the database.

Click 3 – MySQL The third tier is the **MySQL database**, named cloudmonitor, with six tables. The server talks to it through JPA and JDBC.

Click 4 – Outside services There are two outside pieces. **OSHI** reads the real CPU, memory, disk and network of my PC. **Gmail SMTP** sends critical alert emails in the background, so the app never waits for email.

Understand it yourself

Three-tier architecture: Presentation (browser), logic (server) and data (database) are kept separate, so each can be changed without breaking the others.

REST API: URLs the browser calls to read or change data, for example `GET /api/resources` or `POST /api/resources/5/stop`. Data is sent as JSON.

Controller → Service → Repository: Controller = receives HTTP requests. Service = the actual logic (for example: check rules, calculate cost). Repository = saves and loads data. This is called a layered architecture.

SMTP: Simple Mail Transfer Protocol, the standard for sending email. Gmail's SMTP server is used on port 587 with TLS encryption.

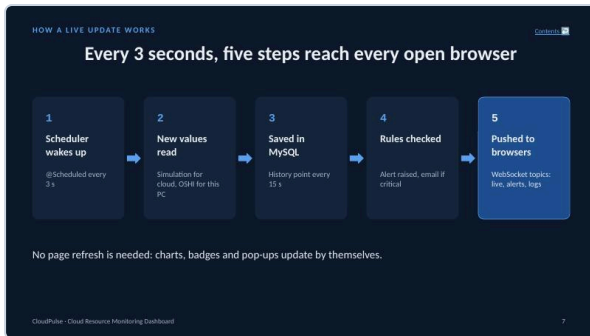
If they ask

Q. What is the difference between REST and WebSocket here?

REST is request-response: the browser asks and the server answers once. WebSocket is a permanent connection, so the server can push new values every 3 seconds without being asked.

Q. Why is email sent in the background?

Sending an email can take a few seconds. If it ran in the main flow, the monitoring would freeze. A separate background thread sends it.



SLIDE 7 OF 15

How a live update works

1.5 min

On the slide: Five steps with arrows: Scheduler wakes up → New values read → Saved in MySQL → Rules checked → Pushed to browsers.

What to say

Click 1 – Step 1 This is the heart of the project, the MonitoringService. Every 3 seconds a **scheduled job** wakes up automatically, using Spring's `@Scheduled` annotation.

Click 2 – Step 2 It **reads new values** for every resource. For the ten cloud servers, the values come from a simulation. For my own PC, they come from OSHI, which reads the real hardware.

Click 3 – Step 3 The latest values are **saved in MySQL**. Every 15 seconds a history point is also stored, which is used to draw the 24-hour charts.

Click 4 – Step 4 Then every **alert rule is checked**. If a value breaks a rule, an alert is raised, and if it is critical, an email is sent.

Click 5 – Step 5 Finally, one message is **pushed over WebSocket** to every open browser, on topics for live data, alerts and logs.

Click 6 – Note That is why nobody needs to refresh the page. Charts, badges and pop-ups update by themselves.

Understand it yourself

@Scheduled: A Spring annotation that runs a method again and again at a fixed interval, like

```
@Scheduled(fixedRate = 3000)
```

 for every 3000 milliseconds.

Why history only every 15 seconds?: Saving every 3 seconds for 11 resources would create too many rows. Every 15 seconds is enough for smooth charts, and old points are removed after 24 hours.

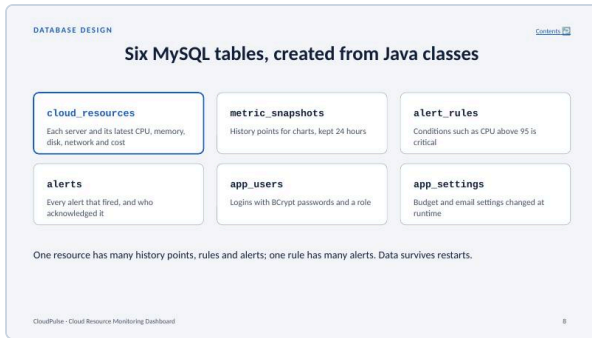
If they ask

Q. What happens if two browsers are open?

Both receive the same message at the same moment, because they are both subscribed to the same topic.

Q. What if the browser loses connection?

Live updates pause until the connection is back. No data is lost, because everything is stored in MySQL; refreshing the page loads it all again.



SLIDE 8 OF 15

Database design

1 min

On the slide: Six table cards: `cloud_resources`, `metric_snapshots`, `alert_rules`, `alerts`, `app_users`, `app_settings`.

What to say

Click 1 – `cloud_resources` The database has six tables. The centre table is **`cloud_resources`**: one row for each server, with its name, provider, region, status and its latest CPU, memory, disk and network values.

Click 2 – `metric_snapshots` and `alert_rules` **`metric_snapshots`** stores history points for the charts, kept for 24 hours. **`alert_rules`** stores conditions such as CPU above 95 percent is critical.

Click 3 – `alerts`, `app_users`, `app_settings` **`alerts`** stores every alert that was raised and who acknowledged it. **`app_users`** stores logins with BCrypt passwords and a role. **`app_settings`** stores the budget and email settings that can change while the app is running.

Click 4 – Note The relationships are simple: one resource has many history points, rules and alerts, and one rule can raise many alerts. The tables are created automatically by Hibernate from my Java entity classes, and data survives restarts.

Understand it yourself

Entity class: A Java class marked with `@Entity`. Hibernate turns each such class into a table and each object into a row.

One-to-many: One row in one table is linked to many rows in another, like one resource → many snapshots. It is done with a foreign key (`resource_id`).

Why MySQL instead of H2?: I started with H2, an in-memory database that is lost when the app stops. I moved to MySQL so data is permanent.

If they ask

Q. Did you normalise the database?

Yes. Resource details are stored once in `cloud_resources`, and other tables only keep the resource's id, so nothing is duplicated.

Q. What problem did you face?

A column named "value" broke table creation because `VALUE` is a reserved word in SQL. I renamed it to `metric_value`.



SLIDE 9 OF 15

Monitoring

1.5 min

On the slide: Four numbers (3 s, 11, 24 h, 89%), a line chart of a daily load pattern, and a short explanation.

What to say

Click 1 – The numbers Here are the key numbers. Data updates every **3 seconds**. There are **11 resources**: ten simulated cloud servers on AWS, Azure and Google Cloud, plus my own PC. Charts can show from 15 minutes up to **24 hours** of history.

And **89 percent**: this was my PC's memory usage in CloudPulse, and Windows Task Manager showed the same value at the same time. So the real data is correct.

Click 2 – The chart Why simulated servers? Connecting real AWS or Azure accounts needs paid subscriptions. So the simulation behaves like real servers: it follows a daily load curve, busiest in the afternoon, with small random changes and occasional spikes. This chart shows that daily pattern.

The eleventh resource is real. OSHI reads my PC's real CPU, memory, disk, network and Windows uptime, and I checked all of them against Task Manager.

Understand it yourself

OSHI: Operating System and Hardware Information, a free Java library that reads CPU, memory, disks, network and uptime from Windows, Linux or macOS.

Simulation: Code that generates realistic values: base load + daily curve + small random noise + rare spikes (about 0.15% chance per reading).

Uptime: How long the machine has been running since its last restart.

If they ask

Q. Is your data fake?

Ten cloud servers are simulated because real cloud accounts cost money, but my PC's data is 100 percent real and was verified against Task Manager. Because the simulation is in one service, it can be replaced by real AWS or Azure APIs.

Q. How does OSHI read the CPU?

It calls the operating system's own counters (on Windows, through system APIs), takes two readings a moment apart, and calculates the percentage of time the CPU was busy.



SLIDE 10 OF 15

Alerts and email

1.5 min

On the slide: An example rule card (CPU > 95%, Critical) and five numbered steps, then a spam-guard line.

What to say

Click 1 – Example rule Alerts are based on rules. Here is an example: if CPU is above 95 percent, it is **Critical**. Admins can add, edit, switch off or delete rules, for all servers or one server.

Click 2 – Step 1 When a rule is broken, first a **pop-up appears instantly** in every open browser.

Click 3 – Step 2 The alert is **saved in MySQL** and listed under Active alerts.

Click 4 – Step 3 It is also **written to the live logs**.

Click 5 – Step 4 If it is critical, an **HTML email** is sent through Gmail to up to five people. The Gmail App password is kept only in environment variables, never in my code or on GitHub.

Click 6 – Step 5 A user then **acknowledges** the alert, one by one or all together, and the user's name is saved, so we know who handled it.

Click 7 – Spam guards To avoid flooding, the same rule fires at most once every 2 minutes per server, and at most 20 emails are sent per hour.

Understand it yourself

Severity: How serious an alert is: for example Warning or Critical. Only Critical sends email.

Gmail App password: A special 16-character password Google gives for apps, instead of your real Gmail password. It is stored as an environment variable (MAIL_PASSWORD).

Environment variable: A setting given to the program by the operating system or IDE when it starts, not written in the code. That keeps secrets out of GitHub.

Acknowledge: Marking an alert as "seen, I am handling it".

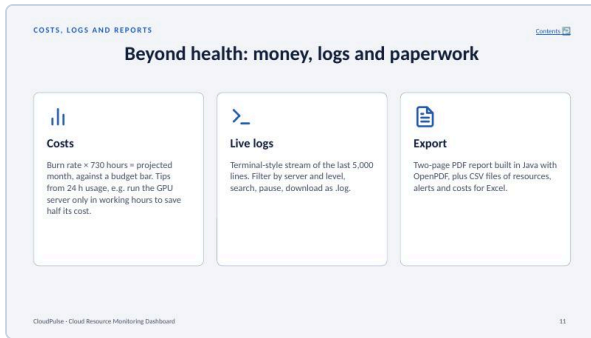
If they ask

Q. Why not send email for every alert?

Too many emails make people ignore them. Warnings are shown on screen; only critical ones go to email.

Q. How do you stop alert spam?

A cooldown of 2 minutes per rule per server, and a limit of 20 emails per hour.



SLIDE 11 OF 15

Costs, logs and reports

1.5 min

On the slide: Three cards: Costs, Live logs, Export.

What to say

Click 1 – Costs The cost page adds up the hourly price of every running server. That is the **burn rate**. Burn rate multiplied by 730 hours gives the **projected monthly cost**. In my demo it was about 3.64 dollars per hour, so about 2,659 dollars a month, compared against a budget of 3,000 that the admin can change.

It also gives **savings tips** from the last 24 hours of usage, for example: this server is mostly idle, run it only during working hours to save half its cost.

Click 2 – Live logs The logs page is like a terminal. It streams the last 5,000 log lines live. You can filter by server and level, search, pause the stream, and download the logs as a file.

Click 3 – Export The export menu creates a two-page **PDF report**, generated in Java using OpenPDF, with resources, alerts and costs. It can also download **CSV** files that open in Excel.

Understand it yourself

Why 730 hours?: Average hours in a month: $365 \text{ days} \times 24 \text{ hours} \div 12 \text{ months} = 730$.

Log level: How important a log line is: INFO (normal), WARN (warning), ERROR (problem).

OpenPDF: A free Java library to create PDF files with text, tables and colours.

If they ask

Q. Are the prices real?

They are fixed hourly prices similar to real cloud prices. With real accounts, the AWS or Azure billing API would give exact costs.

Q. Where are logs stored?

In memory, the last 5,000 lines. Storing them in the database is in my future scope.

SECURITY AND ROLES

Viewers can watch; only admins can change things

Action	Admin	Viewer
See dashboards, logs, costs	Yes	Yes
Acknowledge alerts, export	Yes	Yes
Start, stop, restart servers	Yes	No
Edit alert rules	Yes	No
Change budget and email	Yes	No

- * Passwords stored as BCrypt hashes
- * JWT token on every request, valid 8 hours
- * WebSocket checks the token too
- * Admin rules checked on the server with @PreAuthorize
- * Gmail password only in environment variables

CloudPulse - Cloud Resource Monitoring Dashboard

SLIDE 12 OF 15

Security and roles

1 min

On the slide: A permissions table (Admin / Viewer) and a list of security measures.

What to say

Click 1 – Roles table CloudPulse has two roles. A **Viewer** can see dashboards, logs and costs, acknowledge alerts and export reports. Only an **Admin** can start, stop or restart servers, edit alert rules, and change the budget and email settings.

Click 2 – How it is protected Passwords are stored as **BCrypt hashes**, never as plain text. After login the user gets a **JWT token** valid for 8 hours, which is sent with every request. The WebSocket connection checks the token too.

Most importantly, admin rules are checked **on the server** with `@PreAuthorize`, not just by hiding buttons. So even if a viewer calls the API directly, they get a 403 Forbidden error. And the Gmail password is only in environment variables.

Understand it yourself

BCrypt: A slow, salted hashing method. A hash cannot be reversed to get the password; at login the typed password is hashed again and compared.

JWT (JSON Web Token): A signed token the server gives after login. It contains the username, role and expiry time. The server checks the signature on each request, so no session is stored.

@PreAuthorize: A Spring Security annotation on a method, for example `@PreAuthorize("hasRole('ADMIN')")`. Spring blocks the call if the user is not an admin.

403 vs 401: 401 = not logged in. 403 = logged in but not allowed.

If they ask

Q. What if someone steals the JWT token?

They could use it until it expires, so it lasts only 8 hours. In production I would add HTTPS so tokens cannot be read on the network.

Q. Why BCrypt and not MD5 or SHA?

MD5 and SHA are very fast, so attackers can guess billions of passwords per second. BCrypt is deliberately slow and adds a random salt.

TESTING

All 16 test cases pass after four real bugs were fixed

16/16

manual test cases passed: login, roles, live data, alerts, email, costs, PDF, logs, MySQL, mobile

Bug found	Fix
Column "value" is reserved in SQL	Renamed to metric_value
H2 security rule broke after MySQL	Replaced with a plain path rule
150+ alerts in 15 minutes	Spike chance 0.6% to 0.15%
Email "Not set up"	Trim spaces in variable names

CloudPulse - Cloud Resource Monitoring Dashboard

SLIDE 13 OF 15

Testing

1 min

On the slide: A big 16/16 card and a table of four bugs and their fixes.

What to say

Click 1 - 16/16 I tested every module by hand with **16 test cases**: login, roles, live data, alerts, email, costs, PDF, logs, MySQL and mobile view. All 16 pass.

Click 2 - Bugs fixed Testing found real bugs. First, a column named "value" is a reserved word in SQL, so I renamed it to metric_value. Second, after moving from H2 to MySQL, an old H2 security rule crashed startup, so I replaced it with a simple path rule.

Third, the simulation created over 150 alerts in 15 minutes, so I reduced the spike chance from 0.6 percent to 0.15 percent. Fourth, email showed "Not set up" because of a hidden space in the variable name, so now the code trims spaces.

Understand it yourself

Manual testing: Running the app and checking each feature by hand against an expected result. The full test-case table is in the project report.

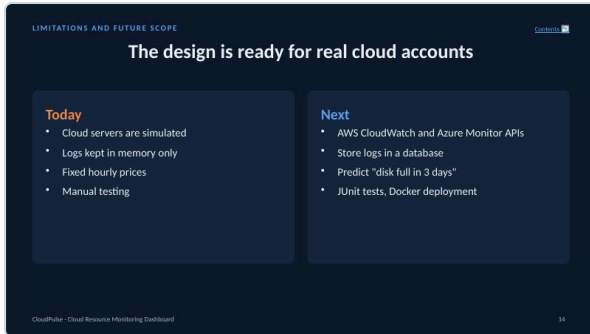
If they ask

Q. Why no automated tests?

Because of time I tested manually; JUnit tests are in my future scope.

Q. Which bug taught you the most?

The email bug: a single invisible space broke it. I learned to log clear setup messages and to clean input values.



SLIDE 14 OF 15

Limitations and future scope

1 min

On the slide: Two cards: Today (limitations) and Next (future work).

What to say

Click 1 – Today To be honest about the limitations: the cloud servers are simulated, logs are kept in memory only, prices are fixed, and testing was manual.

Click 2 – Next In the future: connect the real **AWS CloudWatch and Azure Monitor APIs**. Because the simulation is inside one service, it can be replaced without changing the rest of the project. I would also store logs in the database, add prediction such as "disk full in 3 days", and add JUnit tests and Docker deployment.

Understand it yourself

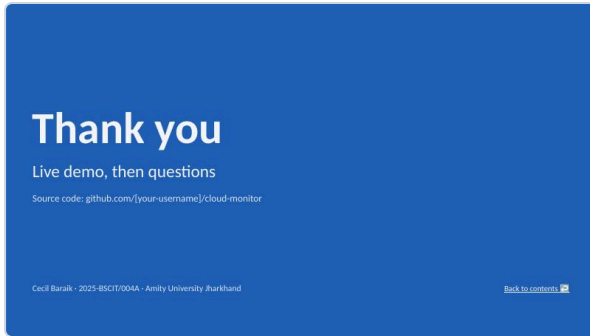
Docker: A tool that packs the app and everything it needs into a container, so it runs the same on any machine.

Prediction: Using the trend of past values (for example disk growth per day) to estimate when a limit will be reached.

If they ask

Q. How much would change to use real AWS?

Only the data source. A new service would call the AWS API instead of the simulation; the database, alerts, WebSocket and frontend stay the same.



Thank you

15 sec, then demo

On the slide: "Thank you", "Live demo, then questions", the GitHub link, your name and roll number.

What to say

Slide opens (animates by itself) Thank you for listening. The source code is on GitHub. Now I will show a short live demo, and then I am happy to answer your questions.

Live demo script (5–7 minutes)

Talk while you click. One sentence per step is enough.

Before the viva	Start MySQL, then run the Spring Boot app in IntelliJ (check the log says <i>Email alerts: ready</i>), then run <code>npm run dev</code> in the frontend folder. Open the app in Chrome and open Task Manager. Log out so you can show the login.
1. Login	Log in as admin. Say: "Passwords are BCrypt hashes, and after login I get a JWT token."
2. Dashboard	Show numbers changing every 3 seconds. Say: "No refresh, this is WebSocket."
3. Real PC	Open your PC's row next to Task Manager. Compare memory and uptime.
4. History chart	Open one resource, switch between 15 min, 1 h and 24 h.
5. Start / stop	Stop a server as admin; show its cost drops. Optionally log in as viewer and show the buttons are not allowed.
6. Alerts and email	Show the rules, edit one to a low limit (for example CPU above 10% critical) so it fires, show the pop-up, then the email in Gmail. Acknowledge it. Change the rule back.
7. Costs	Show burn rate, projected month, budget bar and savings tips.
8. Logs	Show the live stream, filter by ERROR, search, pause.
9. Export	Download the PDF report and open it.

If something fails during the demo: stay calm and say what should happen, then show the same thing in a screenshot from your report. Examiners care more that you understand it than that the Wi-Fi works.